

**Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра
Великого Институт прикладной математики и
информатики**

Курсовой проект по дисциплине «Численные методы»

Отчет по лабораторной работе №5

**Решение задачи Коши для обыкновенных дифференциальных уравнений.
Одношаговые методы**

Выполнил студент гр. 5030102/30004

Зернов.К.В

Преподаватель:

Курц. В.В

Содержание

Формулировка задачи и ее формализация задачи	2
Формулировка задачи	2
Формализация задачи	2
Алгоритм метода	2
Условие применимости	2
Проверка условия	3
Алгоритм модифицированного метода Эйлера ($p = 1$)	3
Предварительный анализ задачи	3
Условие существования решения	3
Тестовый пример	3
Подготовка контрольных тестов	4
Модульная структура программы	4
Численный анализ	7
Графики	7
Вывод	10

Формулировка задачи и ее формализация задачи

Формулировка задачи

Общий вид дифференциального уравнения n -ого порядка задается выражением:

$$F(x, y, y', \dots, y^{(n)}) = 0 \quad (1)$$

где F - некоторая известная функция.

В ряде случаев удастся разрешить уравнение (1) относительно старшей производной:

$$y^{(n)} = f(x, y, y', \dots, y^{(n-1)}) \quad (2)$$

Общее решение уравнения (2) представляет собой n -параметрическое семейство решений $y(x) = y(x, c_1, c_2, \dots, c_n)$. Для получения частного решения необходимо заменить константы $c_1, c_2, \dots, c_n$ на конкретные числовые значения.

Формализация задачи

Задача Коши

$$\begin{cases} y' = f(x, y) & x \in [a, b] \\ y(a) = y_0 \end{cases}$$

В данной лабораторной работе дано дифференциальное уравнение 1-го порядка:

$$y' = \frac{4x+2y}{2x+1}$$

рассматривается отрезок $[0, 4]$.

Необходимо решить задачу Коши с помощью модифицированного метода Эйлера (метод средней точки):

$$\begin{cases} y' = \frac{4x+2y}{2x+1} & x \in [a, b] \\ y(0) = 1 \end{cases}$$

Дано точное решение: $y = (2x + 1) \ln|2x + 1| + 1$

Алгоритм метода

Условие применимости

Функция $f(x, y)$ непрерывна на D и удовлетворяет условию Липшица.

$$\exists L > 0 : |f(x, y_1) - f(x, y_2)| \leq L|y_1 - y_2|,$$

Проверка условия

$f'(x, y)$ - ограничена $\rightarrow f(x, y)$ удовлетворяет условию Липшица

Алгоритм модифицированного метода Эйлера ($p = 1$)

Этот метод решает задачу Коши:

$$\begin{cases} y' = f(x, y) & x \in [a, b] \\ y(a) = y_0 \end{cases}$$

Шаг - $h = \frac{b-a}{N}$, где N - число узлов, b и a - правая и левая граница соответственно.

Формула метода:

$$y_{k+1} = y_k + hf(x_k + \frac{h}{2}, y_k + \frac{h}{2}f(x_k, y_k))$$

Это можно переписать так же как:

$$\begin{cases} y_{k+\frac{1}{2}} = y_k + \frac{h}{2}f(x_k, y_k) \\ y_{k+1} = y_k + hf\left(x_{k+\frac{1}{2}}, y_{k+\frac{1}{2}}\right) \end{cases}$$

где $x_{k+\frac{1}{2}} = x_k + \frac{h}{2}$

Предварительный анализ задачи

Условие существования решения

Функция $f(x, y)$ непрерывна на D и удовлетворяет условию Липшица.

Тестовый пример

Возьмем уравнение:

$$\begin{cases} y' = y \\ y(0) = 1 \end{cases}$$

Точное решение: $y(x) = e^x$

Ошибка по правилу Рунге:

$$\text{Ошибка} \sim \frac{|y_k^{2h} - y_k^h|}{2^s - 1}$$

y^{2h} - решение, полученное с шагов $2h$.

y^h - решение, полученное двумя шагами по h .

$$h = 0.1, 2h = 0.2$$

$$y_1^{2h} = y_0 + 2hf(x_0 + h, y_0 + hf(x_0, y_0)) = 1 + 2 \cdot 0.1f(0.1, 1 + 0.1 \cdot f(0, 1)) = 1 + 2 \cdot 0.1 \cdot 1.1 = 1.22$$

Первый шаг ($0 \rightarrow 0.1$):

$$y_{0.5}^h = y_0 + hf(x_0 + \frac{h}{2}, y_0 + \frac{h}{2}f(x_0, y_0)) = 1 + 0.1 \cdot f(0.05, 1 + 0.05 \cdot 1) = 1 + 0.1 \cdot 1.05 = 1.105$$

$$y_1^h = y_{0.5} + hf(x_{0.5} + \frac{h}{2}, y_{0.5} + \frac{h}{2}f(x_{0.5}, y_{0.5})) = 1.105 + 0.1 \cdot f(0.15, 1.105 + 0.05 \cdot 1.105) = 1.221025$$

у одним шагом ($2h$): 1.22

у двумя шагами (h): 1.221025

Точное значение: 1.2214027582

Погрешность по Рунге: $\frac{|1.22 - 1.221025|}{3} = 0.00034166666666668455 = 3.416670e-04$

Фактическая погрешность: $|1.2214027582 - 1.221025| = 0.0003777582 = 3.777582e-04$

Подготовка контрольных тестов

1. Строим график точного и численного решения, а также график ошибки.
2. Исследование зависимости погрешности от h
Задаем h равное 0.4 и далее уменьшаем его до порядка $1e-4$, в цикле считаем погрешность для локальной и глобальной погрешности
3. Исследовать влияние ошибок в исходных данных на решение
Задаем δ равное 10^{-1} и далее уменьшаем его до порядка 10^{-11} и считаем получившуюся глобальную погрешность.

Модульная структура программы

- `double function(double x, double y);`

Описание: значение производной в точке (x, y)

- ▶ Входные данные:
 - Значение x , в котором считаем значение функции
 - Значение y , в котором считаем значение функции
- ▶ Выходные данные:
 - Значение функции $\frac{4x+2y}{2x+1}$

- `double exactFunction(double x)`

Описание: значение интегральной кривой в точке x

- ▶ Входные данные:
 - Значение x , в котором считаем значение
- ▶ Выходные данные:

– Значение интегральной кривой $(2x + 1) \ln|2x + 1| + 1$

- `double methodEuler(double x_k, double y_k, double h, std::function<double(double, double)> func)`

Описание: считает значение y_{k+1} методом Эйлера

► Входные данные:

- x_k – значение x на шаге k
- y_k – значение y на шаге k
- h – значение шага
- `func` – указатель на функцию y'

► Выходные данные:

- значение y в точке x_{k+h} ($x_k + h$)

- `double modifiedEulerMethod(double x_k, double y_k, double h, std::function<double(double, double)> func)`

Описание: считает значение y_{k+1} модифицированным методом Эйлера

► Входные данные:

- x_k – значение x на шаге k
- y_k – значение y на шаге k
- h – значение шага
- `func` – указатель на функцию y'

► Выходные данные:

- значение y в точке x_{k+h} ($x_k + h$)

- `double rungeRule(int s, double x_k, double y_k, double h, std::function<double(double, double)> func, std::function<double(double, double, double, std::function<double(double, double)>>> method)`

Описание: оценивает погрешность по правилу Рунге

► Входные данные:

- s – порядок метода
- x_k – значение x на шаге k
- y_k – значение y на шаге k
- h – значение шага
- `func` – указатель на функцию y'
- `method` – указатель на метод, которым аппроксимируем решение

► Выходные данные:

- оценка погрешности при шаге h

- `double localError(double yActual, double yMethod)`

Описание: Вычисляет локальную ошибку

► Входные данные:

- `yActual` – точное значение y в точке x_k
- `yMethod` – численное значение y в точке x_{k+1} с подстановкой точного значения в точке x_k

► Выходные данные:

- значение локальной ошибки

- `double globalError(double yActual, double yMethod)`

Описание: Вычисляет глобальную ошибку

► Входные данные:

- `yActual` – точное значение в точке x_k
- `yMethod` – численное значение y в точке x_{k+1} с накопленными ошибками

► Выходные данные:

- значение глобальной ошибки

- `void RunExperiment(
std::string path,
double x0,
double y0,
double right,
std::function<double(double, double)> func,
std::function<double(double)> exactFunc,
std::function<double(double, double, double, std::function<double(double, double)>>> method,
int s)`

Описание: запускает эксперимент для Задачи Коши и записывает все полученные значения в .txt файлы

► Входные данные:

- `path` – путь куда будут сохраняться данные
- x_0 – начальная точка (взято из условия Задачи Коши)
- y_0 – начальное значение в точке x_0 (Задача Коши)
- `right` – правая граница отрезка
- `func` – указатель на функцию y'
- `exactFunc` – указатель на точную функцию интегральной кривой
- `method` – одношаговый метод, которым ищем решение
- `s` – порядок метода

► Выходные данные:

- .txt файлы

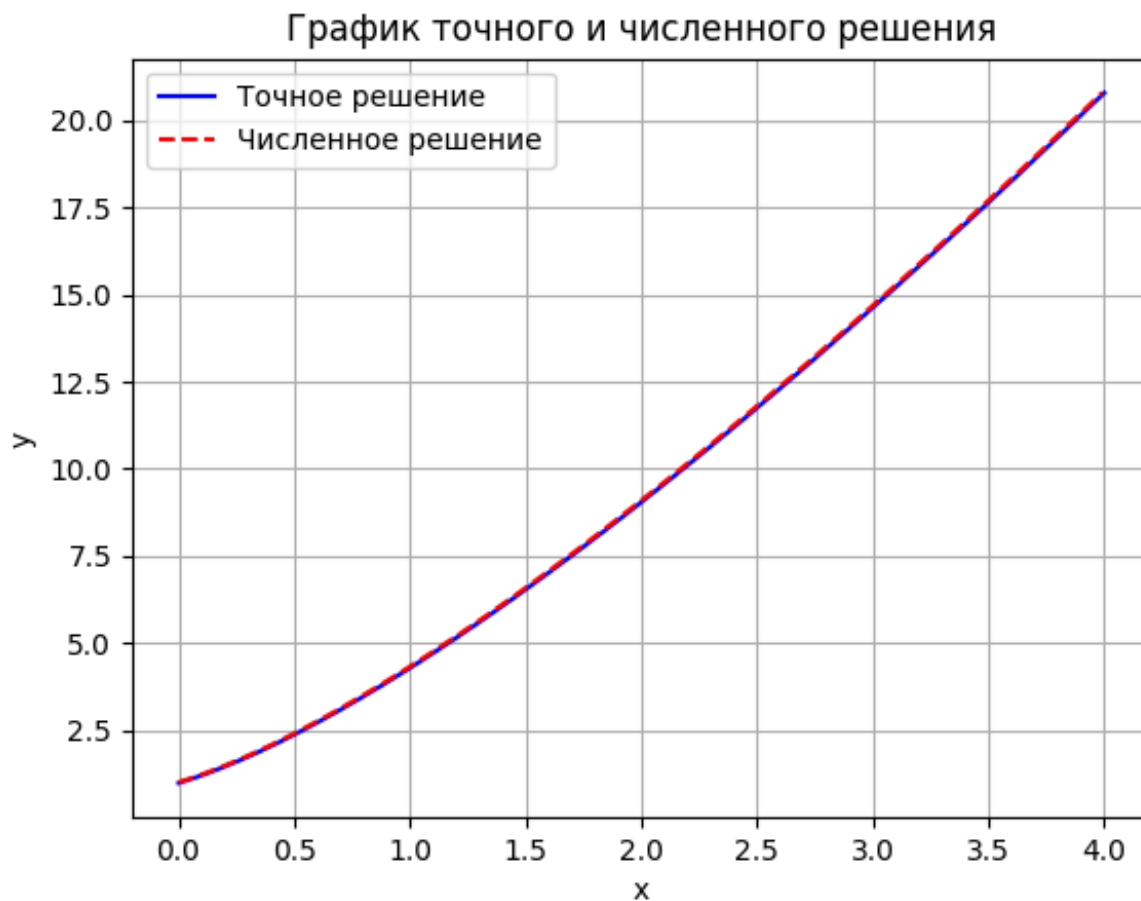
Численный анализ

Исследования, которые были проведены:

1. Исследование зависимости глобальной и локальной погрешности от величины шага.
2. Точность метода
3. Исследование устойчивости метода к небольшим погрешностям в исходных данных

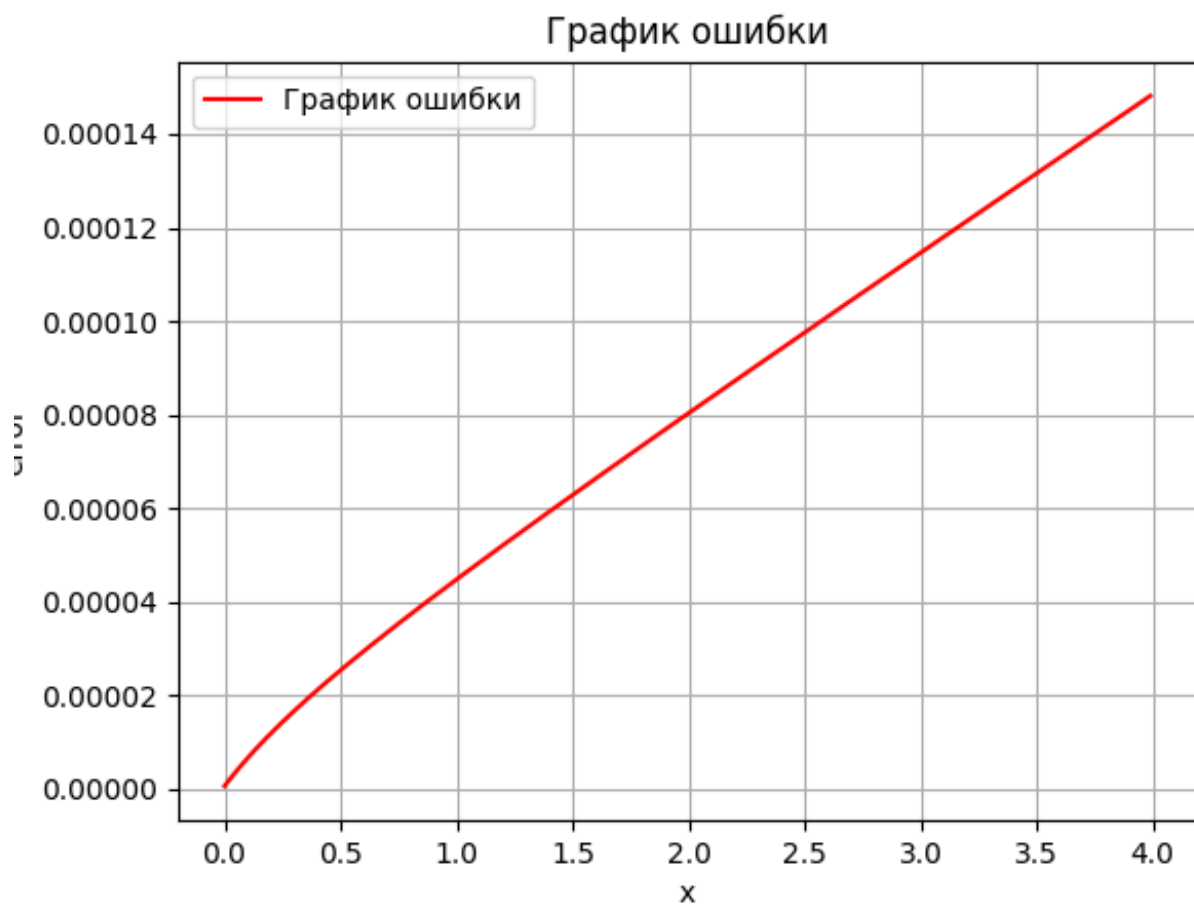
Графики

1. Численное решение модифицированным методом Эйлера ($h = 0.01$)



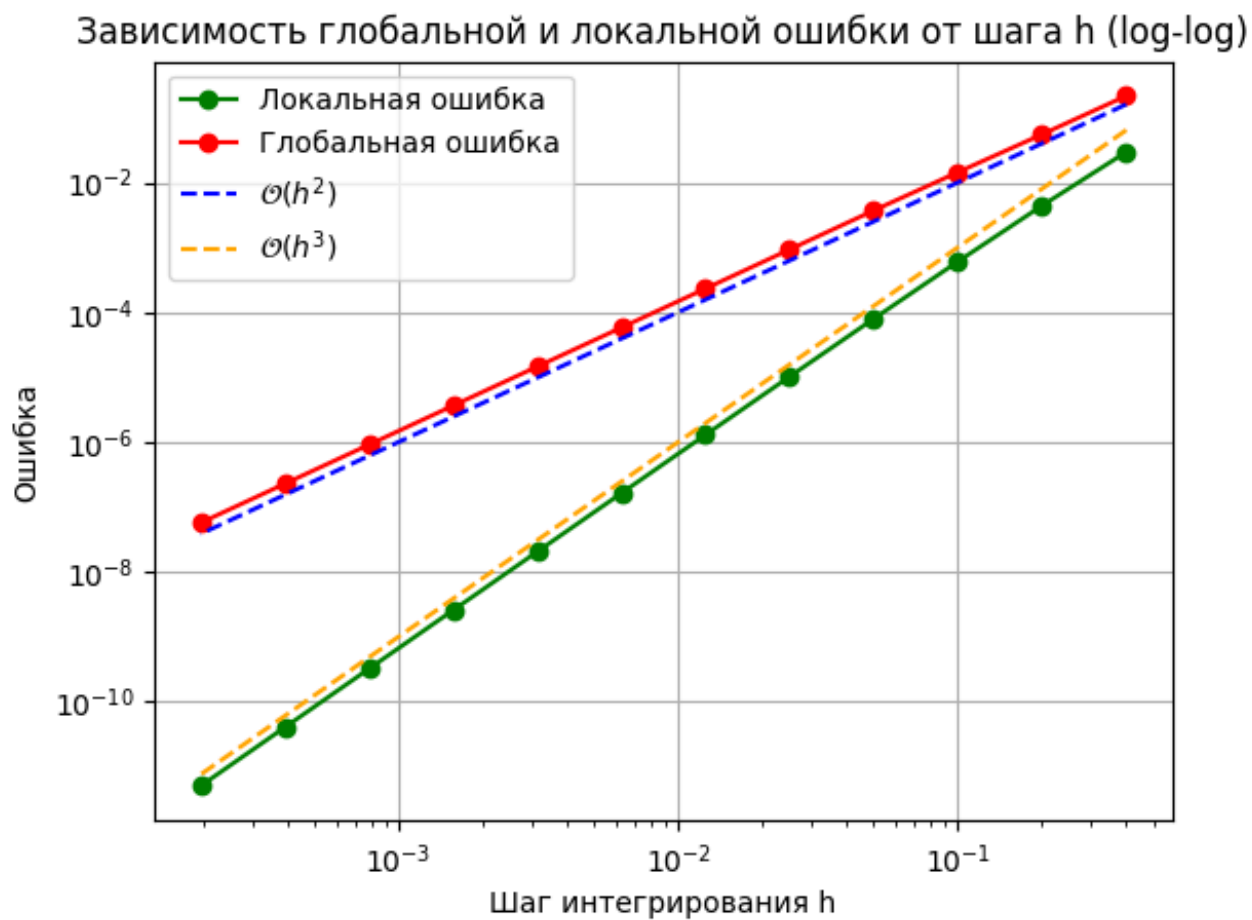
Из графика видно, что численное решение хорошо аппроксимирует точное решение.

2. Ошибка модифицированного метода Эйлера



С увеличением x ошибка растет линейно

3. Зависимость шага от локальной и глобальной погрешности



Из графика заметно, оценки что локальная и глобальная ошибки соответствует теории

4. Зависимость погрешности в начальных на данных на глобальную ошибку



Из графика видно, что модифицированный метод Эйлера является устойчивым

Вывод

Модифицированный метод Эйлера является эффективным алгоритмом для решения задачи Коши ОДУ 1-го порядка. Данный метод обеспечивает вычислительную сложность и точность получаемого решения, демонстрируя сходимость соответствующего порядка точности.